

Keeping Engineers in the Loop: Explainable Requirement Defect Screening Across Software Requirements Specifications

Awaiz Ahmed
Al Ain University
United Arab Emirates
awaiz42ad@gmail.com

Mohammed Umar
Al Ain University
United Arab Emirates

Abstract—Requirements written in natural language frequently contain ambiguity, incompleteness, and statements that are difficult to validate consistently. This experience report presents a human-centered requirement-validation workflow applied to the PURE corpus of 79 publicly available software requirements specification (SRS) documents. From this corpus the workflow extracted 9,165 requirement-like statements as processing context and adjudicated a 110-row reviewed subset that forms the basis of all reported evaluation. On the reviewed subset we compared three screening conditions: the original seed heuristic, a flat rule-based screen, and an explainable precedence-based screen that returns a defect label, rationale, source span, rewrite guidance, and confidence. The seed heuristic achieved 0.573 accuracy and 0.499 macro-F1, the flat rule screen achieved 0.500 accuracy and 0.478 macro-F1, and the explainable screen achieved 0.855 accuracy and 0.891 macro-F1. The main finding is that explanation-oriented screening substantially improves practical review support on this subset, while the remaining errors are dominated by formatting artifacts, truncated list boundaries, and contextual judgments that still require human reviewers. The contribution is not autonomous requirement assessment; it is an auditable human-in-the-loop workflow for prioritizing requirement-quality review. To support replication, the reviewed gold set, the frozen annotation guide, the evaluation script, and the per-class confusion data are released as a supplementary package.

Index Terms—requirements engineering, human-centered AI, explainable AI, requirement validation, requirements quality, requirements smells, human-in-the-loop review

I. INTRODUCTION

Requirements engineering is a human-centered and socio-technical activity. Engineers, analysts, and stakeholders use natural language to negotiate intended system behavior, constraints, interfaces, and acceptance expectations. Because these artifacts guide design, implementation, testing, procurement, and maintenance, defects in requirements can propagate into later lifecycle stages where correction is more expensive.

Natural-language requirements are especially difficult to validate at scale. A statement may look syntactically well formed while still being ambiguous, missing an acceptance condition, or difficult to test. Human review is therefore essential, but large legacy specifications make exhaustive inspection slow and inconsistent. This motivates assisted review

workflows that can triage likely defects without replacing the reviewer.

This paper reports an experience-oriented empirical study of that problem. The workflow extracts requirement-like statements from heterogeneous files, constructs a reviewed subset, and compares three screening conditions. The design emphasizes explainability: the strongest condition does not only return a label, but also a rationale, source span, rewrite guidance, and confidence score. This work addresses the challenge of keeping humans in control as AI and automation enter requirements engineering workflows.

This paper makes three contributions. First, it documents a staged extraction and curation pipeline over the public PURE corpus of requirements documents [3], from which 9,165 requirement-like statements were extracted as processing context and a 110-row reviewed subset was adjudicated and released as an evaluation benchmark. The extracted-statement count describes the processing scope of the pipeline, whereas all reported accuracy is measured on the reviewed subset. Second, it grounds and applies a compact defect taxonomy covering ambiguity, incompleteness, non-testability, and clean requirements in established requirements-quality characteristics. Third, it evaluates a human-in-the-loop explainable screening condition against two simpler baselines on the reviewed subset and analyses the error patterns that remain for human reviewers.

II. RELATED WORK AND MOTIVATION

Requirements quality and standards. Requirements quality has long been connected to clarity, completeness, consistency, and verifiability. ISO/IEC/IEEE 29148 positions requirements engineering as a lifecycle process and defines characteristics of well-formed requirements, including that individual requirements should be unambiguous, complete, and verifiable, so that they support later verification and validation [1]. These characteristics provide the vocabulary against which automated screening is usually judged, and they directly motivate the defect taxonomy used in this study (Section V).

Automated detection of requirement defects. A long line of work has tried to surface quality problems in natural-language requirements automatically. QuARS introduced a quality model and tool that flag linguistic defect indicators such as vagueness, optionality, and underspecification for reviewer attention [4]. Requirements smells extended this idea with lightweight, operationalized detectors and showed that automated screening helps reviewers locate quality problems while still requiring human interpretation [2]. Broader surveys of natural-language processing for requirements engineering map defect and quality analysis as a recurring task family and highlight the persistent difficulty of evaluation and replication [5], and recent perspectives revisit automated requirements-quality analysis and its open problems [6]. A common thread is that these tools return a flag or a category; the reviewer is left to reconstruct *why* a statement was flagged.

AI-assisted and LLM-based requirements engineering. Large language model (LLM) and AI-assisted requirements engineering work renews this direction. Surveys and guidelines report opportunities in elicitation, analysis, simplification, traceability, and validation, but also identify challenges around reproducibility, trust, data access, and user-centered deployment [7]–[9]. A recent systematic literature review of LLMs for requirements engineering further observes that most current studies concentrate on generative activities such as elicitation and validation rather than on defect detection and classification, which dominated earlier NLP-based work [10]. These concerns are central to this work because requirements decisions are accountable human decisions, not merely text-classification tasks.

Explainability and decision support. Research on explanation in artificial intelligence argues that useful explanations are contrastive and selective, and that they should mirror how people actually justify decisions rather than simply expose model internals [11]. This framing motivates the reviewer-facing output evaluated here: a label alone is weak evidence, but a label paired with the triggering phrase, a short rationale, and a concrete rewrite gives the reviewer something to accept, reject, or correct.

Positioning. Compared with earlier defect detectors that emit a category or a rule hit [2], [4], and with recent LLM4RE work that largely targets generation tasks [10], the present study focuses on the reviewer-facing output as the unit of value: an auditable, explanation-oriented screen used for triage over a public, heterogeneous corpus, with the human retained as the decision-maker. The remainder of the paper develops and evaluates this explainable condition.

III. RESEARCH QUESTIONS

RQ1. How accurately can an explainable screening workflow detect ambiguity, incompleteness, non-testability, and clean requirements in a reviewed SRS subset?

RQ2. How does the explainable screening condition compare with the original seed heuristic and a flat rule-based screen?

TABLE I
CORPUS OVERVIEW

Metric	Value
Source corpus	PURE [3]
Documents processed	79
Requirement-like statements (scope)	9,165
Reviewed gold-set rows (evaluation)	110
Candidate contradiction pairs	220
Document formats	PDF, DOC, HTML, HTM, RTF

RQ3. What error patterns remain after explainable screening, and what do they imply for human-in-the-loop requirements review?

IV. CORPUS AND EXTRACTION PIPELINE

The study draws on PURE, a public dataset of 79 natural-language requirements documents collected from the web and spanning multiple domains and levels of abstraction [3]. Using a public corpus makes the processing stage replicable and avoids the proprietary-data limitations common in this area. The documents are supplied in PDF, DOC, HTML, HTM, and RTF formats. A staged extraction workflow was required because no single parser handled the full archive reliably. PDFs were processed through text extraction, DOC files through document-conversion utilities, and markup-based files through format-specific parsers. The goal was not perfect document reconstruction; it was to build a review-oriented corpus suitable for triage and analysis.

Requirement-like statements were extracted when they contained normative modal language such as *shall*, *must*, or *should*, or when they matched likely actor-action requirement patterns. The resulting dataset stores document identity, requirement identity, extracted text, word count, extraction confidence, weak lexical cues, incompleteness cues, seeded labels, human-reviewed labels, and predicted labels from each screening condition.

Table I summarizes the corpus. The extraction produced 9,165 requirement-like statements as processing context for the study; these are not all individually adjudicated, and all reported accuracy is computed on the 110-row reviewed subset described in Section V. The median document contributed 52 extracted statements, while the largest contributed 1,089, confirming that review support must handle both small and large specifications.

V. DEFECT TAXONOMY AND REVIEWED SET

A. Selection and grounding of the defect labels

The taxonomy is deliberately compact and is grounded in the well-formedness characteristics of ISO/IEC/IEEE 29148 [1]. Three of those characteristics are both highly consequential for downstream verification and validation and reliably observable in text, which makes them suitable targets for triage: *unambiguous*, *complete*, and *verifiable*. The taxonomy maps one defect label to each

TABLE II
DEFECT LABEL, STANDARD CHARACTERISTIC, AND SIGNAL

Label	29148 characteristic	Representative signal
Ambiguity	Unambiguous	Vague or subjective terms (e.g., <i>secure</i> , <i>adequate</i> , <i>broad support</i>)
Incompleteness	Complete	Trailing colons, placeholders, truncated clauses
Non-testability	Verifiable	Claims with no measurable or observable acceptance condition
Clean	Meets the above	Specific, bounded, verifiable as written

(ambiguity, incompleteness, and non-testability), plus a *clean* label for statements that satisfy all three as written. Table II states this mapping together with a representative textual signal for each label. These attributes also coincide with defect families that earlier automated detectors target, namely linguistic vagueness and underspecification in QuARS [4] and the corresponding requirements smells [2], which supports both their relevance and their detectability.

Consistency, the remaining commonly cited characteristic, was deliberately excluded from the primary single-label scheme because it requires cross-statement reasoning rather than a judgment about a single requirement. It is instead handled separately as 220 candidate contradiction pairs (Table I), which are flagged for review but not scored in this study.

B. Label definitions

The study uses one primary label per requirement statement. *Clean* means the statement is specific enough to review and does not show a primary defect under the taxonomy. *Ambiguity* covers vague, subjective, or multiply interpretable wording such as *appropriate*, *adequate*, *easily*, *secure*, *intuitive*, or broad uses of *support*. *Incompleteness* covers missing conditions, unfinished list introductions, placeholders, truncated sentence endings, and incomplete extracted fragments. *Non-testability* covers statements that do not express an objectively verifiable system behavior as written.

C. Reviewed subset and annotation procedure

A 110-row curated subset was adjudicated by a single reviewer. The subset was designed to preserve defect diversity rather than mirror the full-corpus frequency distribution. To make the procedure auditable, annotation followed a written guide that fixes the unit of analysis, the label definitions and signals, illustrative examples, and a reviewer workflow; each change to a machine-seeded label was recorded with a short justification in a review-notes field. The guide was frozen before adjudication and is released with the supplementary package. Table III reports the reviewed label distribution. The non-testability count is low because several machine-seeded non-testability rows became clean or ambiguous after review; this reveals that simple checks often confuse missing

TABLE III
REVIEWED GOLD-SET COMPOSITION

Label	Rows	Share
Clean	40	36.4%
Ambiguity	29	26.4%
Incompleteness	40	36.4%
Non-testability	1	0.9%

TABLE IV
REPRESENTATIVE REVIEWED EXAMPLES AND EXPLANATIONS

Label	Statement, span, and rationale
Clean	<i>"The Host OS shall support a hierarchical, tree-structured file system."</i> Span: none. Rationale: no primary defect under the taxonomy.
Ambiguity	<i>"... the store of access permissions ... must be secure."</i> Span: <i>secure</i> . Rationale: context-dependent; needs measurable bounds.
Incompleteness	<i>"If transmission status indicates an error ..., the following actions shall"</i> Span: trailing boundary. Rationale: clause and object are missing.
Non-testability	<i>"Should one of these gates be lowered ... the results could be catastrophic."</i> Span: full statement. Rationale: no objectively verifiable behavior is stated.

measurement signals with genuinely untestable requirements. The single-reviewer design and the resulting class imbalance are revisited in Section X.

Table IV gives one representative reviewed example per label, together with the source span and rationale emitted by the explainable condition, to make the labels and the explanation format concrete.

VI. SCREENING CONDITIONS

Three conditions were evaluated.

Seed heuristic. The first condition uses the original machine-seeded labels produced during corpus construction. It is intentionally simple and depends heavily on weak lexical cues and extraction signals.

Flat rule screen. The second condition applies a direct rule set. It prioritizes obvious incompleteness markers such as trailing colons, placeholders, and truncated endings; then flags common vague terms; then uses a conservative non-testability rule; otherwise it predicts clean.

Explainable precedence-based screen. The third condition is a deterministic precedence-based screen that emits reviewer-facing explanations. It prioritizes extraction and list-boundary problems before ambiguity, because incomplete fragments can otherwise be misclassified as vague requirements. For each statement it returns a primary label, source span, rationale, rewrite guidance, and confidence. Table V shows the output schema. Because the screen is rule-based rather than trained, it requires no labeled training data; the structured-output schema and an accompanying prompt template are released so that the

TABLE V
EXPLAINABLE OUTPUT SCHEMA

Field	Purpose
Defect label	Primary prediction used for evaluation
Rationale	Reviewer-facing reason for the decision
Source span	Phrase or boundary that triggered the flag
Rewrite guidance	Direction for improving the statement
Confidence	Triage signal, not a replacement for review

TABLE VI
MODEL-COMPARISON RESULTS ON THE REVIEWED SUBSET

Condition	Acc.	Prec.	Rec.	F1
Seed heuristic	0.573	0.586	0.685	0.499
Flat rule screen	0.500	0.654	0.621	0.478
Explainable precedence	0.855	0.911	0.891	0.891

same reviewer-facing contract can be instantiated with an LLM in future work.

VII. RESULTS

Table VI reports the completed comparison on the 110-row reviewed subset. The seed heuristic achieved 0.573 accuracy and 0.499 macro-F1. The flat rule screen achieved 0.500 accuracy and 0.478 macro-F1. The explainable precedence-based screen achieved the strongest result, with 0.855 accuracy and 0.891 macro-F1. Fig. 1 shows the macro-F1 comparison.

Table VII reports per-class results for the explainable condition. The condition performed best for ambiguity and non-testability, while the clean and incompleteness labels show the remaining tension between valid but broad requirements and truncated extraction artifacts. The single non-testability case should not be over-interpreted; it is retained because it shows why rare defect classes need targeted sampling in future work. Fig. 2 shows the reviewed defect distribution.

VIII. QUALITATIVE ERROR ANALYSIS

The strongest qualitative pattern was that document formatting and requirement quality are entangled. A trailing colon, a placeholder such as *xxx*, or a sentence fragment ending after *to aid in* often indicates an extraction boundary rather than an authoring decision. However, the reviewer still cannot validate the row as written. The explainable condition handled many of these cases by explicitly marking the boundary as the source span.

A second pattern was ambiguous capability language. Terms such as *support*, *adequate*, *appropriate*, *secure*, *intuitive*, and *fast enough* frequently require domain-specific thresholds or externally defined criteria. The source-span output is valuable here because it directs the reviewer to the exact phrase that must be bounded.

A third pattern was over-flagging of testability. Simple rules often classified statements such as “the system shall allow an administrator to create a user” as non-testable because they lacked explicit performance thresholds. Human review

TABLE VII
PER-CLASS PERFORMANCE OF THE EXPLAINABLE SCREEN

Label	Support	Prec.	Rec.	F1
Clean	40	0.750	0.975	0.848
Ambiguity	29	0.893	0.862	0.877
Non-testability	1	1.000	1.000	1.000
Incompleteness	40	1.000	0.725	0.841

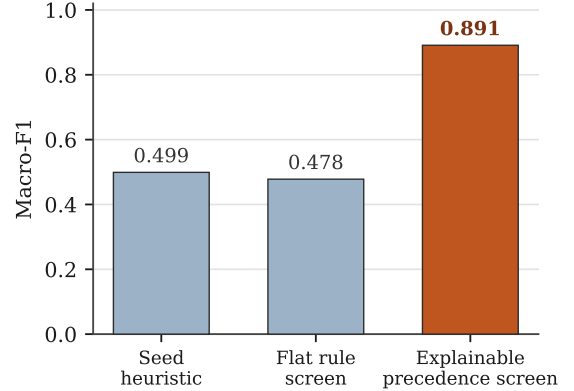


Fig. 1. Macro-F1 by screening condition on the reviewed subset.

treated many such statements as clean functional requirements because the action is objectively observable. This explains why the original seed heuristic produced many false non-testability positives.

IX. DISCUSSION

The results support the value of explanation-oriented screening. The improvement does not come only from adding more rules; it comes from using a review structure that mirrors how engineers inspect requirements [11]. A reviewer needs the defect type, the phrase or boundary that triggered the decision, and a rationale that can be accepted, rejected, or corrected.

The strongest practical finding is that extraction quality and validation quality cannot be separated. Many hard cases involved truncated list introductions, sentence fragments, placeholders, or merged formatting artifacts. These cases are not pure semantic defects, but they still affect the review process because they prevent a requirement from being validated as written.

The second finding is that ambiguity remains highly actionable review work. Vague capability terms need context before they can become testable acceptance criteria. The explainable workflow helps by isolating the relevant span rather than forcing reviewers to inspect an entire document.

The third finding concerns appropriate reliance. The workflow should be used for triage and review support, not autonomous acceptance or rejection of requirements. The human reviewer remains responsible for final judgment, especially when a vague term is acceptable because it is defined elsewhere, or when a requirement references an external standard.

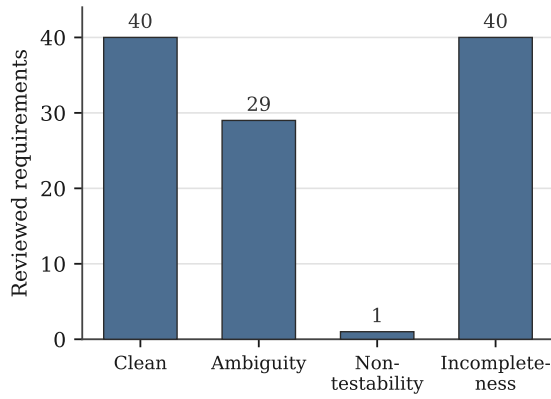


Fig. 2. Reviewed subset distribution after single-reviewer adjudication.

X. THREATS TO VALIDITY

Single-reviewer labeling. The reviewed subset was adjudicated by one reviewer, which introduces subjectivity and possible annotation bias and limits the reliability of the labels. We mitigated this with a frozen annotation guide, recorded per-row review notes, and a diversity-preserving curation strategy, but these do not replace independent annotation. A stronger study should add at least one further annotator, report inter-rater agreement (for example Cohen’s kappa or Krippendorff’s alpha), and seek validation from practicing requirements engineers.

No practitioner evaluation. The study measures offline screening accuracy and the structure of the explanations. It does not measure how practicing requirements engineers perceive or act on the explanations during real review. A user-centered evaluation is needed to establish whether the rationale, source span, and rewrite guidance change review speed, agreement, or trust.

Within-corpus evaluation. The comparison was performed on a curated subset of a single, although public and multi-domain, corpus. The results should not be interpreted as cross-domain generalization.

Extraction noise. Legacy formats, line wraps, tables, and list structures caused fragmentation. Some defects reflect extraction artifacts rather than authoring defects.

Rare-class coverage. Non-testability has a support of one after review, so its per-class metric is unstable and should be read as illustrative only. Because the screen is rule-based, this is a limitation of evaluation coverage rather than of training data. Future work should use targeted sampling to build a more balanced evaluation set for rare defect classes.

Method scope. The explainable screen is a deterministic screening condition with structured explanations. It does not establish that a particular commercial or open-source LLM generalizes across all requirements engineering tasks.

XI. CONCLUSION

This paper presented a human-centered workflow for explainable requirement defect screening across heterogeneous SRS documents. Over the public PURE corpus of 79 documents the workflow extracted 9,165 requirement-like statements as processing context, curated and adjudicated a 110-row reviewed subset, and compared three screening conditions on that subset. The explainable precedence-based screen produced the strongest result, achieving 0.855 accuracy and 0.891 macro-F1.

The core conclusion is narrow but useful: explainable screening can improve practical requirements review when it provides a label, rationale, source span, and rewrite guidance, while preserving human authority over final decisions. The remaining work is to extend the reviewed set, add independent annotators and report inter-rater agreement, obtain validation from requirements-engineering experts, increase coverage of rare defect classes through targeted sampling, instantiate the released explanation schema with an LLM, and evaluate the workflow with practicing requirements engineers in a realistic review setting.

DATA AVAILABILITY

To support replication, the reviewed gold set, the frozen annotation guide, the evaluation script, the LLM prompt template, the per-class confusion matrices, and the corpus inventory are released as a supplementary package accompanying this paper.

REFERENCES

- [1] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, 2nd ed., 2018.
- [2] H. Femmer, D. Méndez Fernández, S. Wagner, and S. Eder, “Rapid quality assurance with requirements smells,” *J. Syst. Softw.*, vol. 123, pp. 190–213, 2017.
- [3] A. Ferrari, G. O. Spagnolo, and S. Gnesi, “PURE: A dataset of public requirements documents,” in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, 2017, pp. 502–505.
- [4] G. Lami, “QuARS: A tool for analyzing requirements,” *Softw. Eng. Inst., Carnegie Mellon Univ., Pittsburgh, PA, USA, Tech. Rep. CMU/SEI-2005-TR-014*, 2005.
- [5] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E.-V. Chioasca, and R. T. Batista-Navarro, “Natural language processing for requirements engineering: A systematic mapping study,” *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–41, 2021.
- [6] D. Porter, J. F. DeFranco, and P. Laplante, “Requirements specification automated quality analysis: Past, present, and future,” *Computer*, vol. 58, no. 1, pp. 101–104, 2025.
- [7] J. J. Norheim, E. Rebentisch, D. Xiao, L. Draeger, A. Kerbrat, and O. L. de Weck, “Challenges in applying large language models to requirements engineering tasks,” *Design Science*, vol. 10, e16, 2024.
- [8] Y. Uygun and V. Momodu, “Local large language models to simplify requirement engineering documents in the automotive industry,” *Prod. Manuf. Res.*, vol. 12, no. 1, 2024.
- [9] A. Vogelsang and J. Fischbach, “Using large language models for natural language processing tasks in requirements engineering: A systematic guideline,” *arXiv:2402.13823*, 2024.
- [10] M. A. Zadenoori, J. Dąbrowski, W. Alhoshan, L. Zhao, and A. Ferrari, “Large language models (LLMs) for requirements engineering (RE): A systematic literature review,” *arXiv:2509.11446*, 2025.
- [11] T. Miller, “Explanation in artificial intelligence: Insights from the social sciences,” *Artif. Intell.*, vol. 267, pp. 1–38, 2019.